

# DBMON-06: Dashboards and Alerting

---

**Series:** DBMON — Database Monitoring | **Notebook:** 6 of 7 | **Created:** March 2026 |  
**Last Updated:** 08/27/2026

## Overview

---

This notebook covers building database monitoring dashboards and configuring alerting for database health. You will learn dashboard design patterns for database KPIs, how to create alert-ready queries for slow query detection, connection pool thresholds, error rate monitoring, and database-specific SLO definitions. These queries are designed to be directly usable in **Dashboards (new)** tiles, **Davis Anomaly Detectors** in Workflows (the modern alerting path), and legacy metric events.

---

## Table of Contents

---

1. [Database KPIs](#)
  2. [Health Overview Dashboard](#)
  3. [Response Time Monitoring](#)
  4. [Error Rate Alerting](#)
  5. [Throughput and Capacity](#)
  6. [Slow Query Alerting](#)
  7. [Database SLO Definitions](#)
  8. [Summary](#)
-

# Prerequisites

| Requirement           | Details                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS or Managed with Grail enabled                                                                       |
| OneAgent              | Deployed on application hosts with database clients                                                      |
| Permissions           | <code>storage:spans:read</code> , <code>storage:metrics:read</code> , <code>storage:entities:read</code> |
| Data                  | Active database traffic across monitored services                                                        |
| Prior Reading         | DBMON-01 through DBMON-05 for context on database monitoring techniques                                  |

## 1. Database KPIs

Effective database monitoring revolves around a small set of key performance indicators. Each KPI maps to a specific dashboard tile and potential alert condition.

| KPI                 | What It Measures                     | Dashboard Tile       | Alert Threshold         |
|---------------------|--------------------------------------|----------------------|-------------------------|
| Response Time (P95) | Query latency at the 95th percentile | Timeseries chart     | > 500ms sustained       |
| Error Rate          | Percentage of failed database calls  | Single value + trend | > 1% of total calls     |
| Throughput          | Queries per second                   | Timeseries chart     | Sudden drop > 50%       |
| Slow Query Count    | Queries exceeding threshold          | Counter + list       | > 10 per 5min window    |
| Connection Errors   | Timeout and refused connections      | Counter              | Any occurrence          |
| Top Queries by Time | Highest-impact query patterns        | Table                | Total time > SLO budget |

## 2. Health Overview Dashboard

---

The health overview provides a single-pane summary of all database systems. Use these queries as dashboard tiles for an executive-level view.

```
// Dashboard tile: Database health summary – one row per database system
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    total_calls = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    error_count = countIf(span.status_code == "error"),
    slow_count = countIf(duration > 500ms),
    unique_databases = countDistinct(db.namespace)
}, by:{db.system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
toDouble(total_calls)) * 100, decimals:2)
| fieldsAdd slow_rate_pct = round((toDouble(slow_count) /
toDouble(total_calls)) * 100, decimals:2)
| sort total_calls desc
```

```
// Dashboard tile: Total database calls – single value
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize total_db_calls = count()
```

```
// Dashboard tile: Database calls per service – identify heaviest consumers
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
}, by:{dt.entity.service}
| fieldsAdd service_name = entityName(dt.entity.service,
type:"dt.entity.service")
| sort call_count desc
| limit 10
```

## Health Score as a Prioritization Signal

The queries above build a custom health overview from spans. If the **Dynatrace Database App** is enabled (GA as of August 2026 for PostgreSQL and MySQL), each monitored database instance also carries a built-in **Health Score** (0–100, combining availability, performance, configuration, and resource-usage signals) — a faster first pass for "which database needs attention right now" than scanning this dashboard row by row.

Use the two together rather than choosing one: the Health Score triages *which* instance to look at, and the span-based queries in this section remain the way to build custom dashboard tiles, thresholds, and SLOs the app's UI doesn't expose.

## 3. Response Time Monitoring

---

Response time monitoring is the most critical aspect of database dashboards. These queries provide both real-time and trend views.

```
// Dashboard tile: Database response time trend by system (6-hour view)
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries p95_ms = percentile(duration / 1ms, 95),
                  by:{db.system},
                  interval:5m
```

```
// Dashboard tile: Response time by database instance (server.address)
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(server.address)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms
}, by:{db.system, server.address, db.namespace}
| sort p95_ms desc
| limit 15
```

```
// Dashboard tile: P50 vs P95 vs P99 comparison – all databases combined
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries p50_ms = percentile(duration / 1ms, 50),
                  p95_ms = percentile(duration / 1ms, 95),
                  p99_ms = percentile(duration / 1ms, 99),
                  interval:5m
```

## 4. Error Rate Alerting

Database errors (connection timeouts, deadlocks, constraint violations) should trigger alerts when they exceed normal baseline levels.

```
// Alert query: Database error rate per 5-minute window
fetch spans, from:-1h
| filter isNotNull(db.system)
| makeTimeseries total = count(),
                  errors = countIf(span.status_code == "error", default:0),
                  by:{db.system},
                  interval:5m
```

```
// Alert query: Error breakdown by type – classify error categories
fetch spans, from:-1h
| filter isNotNull(db.system) and span.status_code == "error"
| summarize error_count = count(),
              by:{db.system, server.address, span.status_message}
| sort error_count desc
| limit 20
```

```
// Alert query: Error rate trend over 24 hours – detect escalating problems
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries total = count(),
                  errors = countIf(span.status_code == "error", default:0),
                  interval:30m
```

## Recommended Alert Thresholds

| Condition                      | Severity | Alert When     |
|--------------------------------|----------|----------------|
| Error rate > 5% for 5 minutes  | Critical | Immediate page |
| Error rate > 1% for 15 minutes | Warning  | Notification   |
| Any connection refused error   | Critical | Immediate page |
| Deadlock detected              | Warning  | Notification   |

## Alerting on Engine Internals — SQL Server Extension Metrics

The alert queries above are span-based (caller-side). If the **Microsoft SQL Server extension** is deployed (see DBMON-02 §6 for the feature sets), the engine's own health signals become alertable too — as **metric events** or **Davis anomaly detectors** on the `sql-server.*` keys, in the same problem/notification pipeline:

| Signal                     | Metric / stream                                            | Starting point                                                        |
|----------------------------|------------------------------------------------------------|-----------------------------------------------------------------------|
| Transaction log filling up | <code>sql-server.databases.log.percentUsed</code>          | Metric event at > 80%, warn at > 70%                                  |
| Engine blocking            | <code>sql-server.general.processesBlocked</code>           | Davis anomaly detector (baseline beats a static count)                |
| Always On degradation      | <code>sql-server.always-on.ag.synchronizationHealth</code> | Metric event on any drop below healthy                                |
| Agent job failures         | <code>failed_jobs / current_jobs</code> <b>log streams</b> | DQL log alert — the failure message text is part of the alert context |

Thresholds carried over from homegrown scripts should be revalidated rather than copied — e.g., scripts typically count *blocking* SPIDs while the extension metric counts *blocked* processes. The decision framework for migrating a script/Telegraf estate onto these signals is in **FAQ-14**.

Log-space alert query — databases whose transaction log crossed 80% in the window:

**Sources:** [Microsoft SQL Server extension \(DT docs\)](#) — the `sql-server.*` metric keys and the feature sets that emit them. **Derived:** the starting-point thresholds in the right-hand column are this series' recommendations, not published values — tune them against your own baseline before alerting on them.

```
// Alert query: transaction logs above 80% utilization (SQL Server extension)
timeseries logUsedPct = avg(`sql-server.databases.log.percentUsed`), from:-24h
| fieldsAdd worst = arrayMax(logUsedPct)
| filter worst > 80
```

```
// Alert query: failed SQL Server Agent jobs with failure context (Jobs
feature set)
fetch logs, from:-24h
| filter isNotNull(job_name) and last_run_outcome == "Failed"
| summarize {failures = count(), latest = max(timestamp)}, by:{job_name,
server}
| sort failures desc
```

## 5. Throughput and Capacity

Monitoring query throughput helps detect traffic anomalies and plan for capacity. A sudden drop in throughput may indicate an upstream failure, while a spike may indicate a runaway process.

```
// Dashboard tile: Queries per minute by database system
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries qpm = count(), by:{db.system}, interval:1m
```

```

// Field names corrected 08/12/2026 – pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and every one of them is null on Grail spans. They fail
SILENTLY: a filter on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
// db.operation -> db.operation.name (stable; set on 50,379 of
57,295 db spans)
// db.statement -> db.query.text (stable; 33,463)
// db.mongodb.collection-> db.collection.name (stable; 6,912)
// db.name -> db.namespace (stable; 57,281)
// Confirm the catalog for your tenant with:
// fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Dashboard tile: Operation mix over time – read vs write trend
fetch spans, from:-6h
| filter isNotNull(db.system) and isNotNull(db.operation.name)
| fieldsAdd op_type = if(
    in(db.operation.name, {"SELECT", "find", "Query", "GetItem", "ReadItem",
"GET", "HGET", "get", "search"}),
    then:"READ",
    else:"WRITE")
| makeTimeseries op_count = count(), by:{op_type}, interval:5m

```

```

// Dashboard tile: Hourly query volume comparison – today vs yesterday
// (Restructured for clean 2-row output: each row has 'period' label +
'count')
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize count = count()
| fieldsAdd period = "today"
| append [
    fetch spans, from:-48h, to:-24h
    | filter isNotNull(db.system)
    | summarize count = count()
    | fieldsAdd period = "yesterday"
]
| fields period, count

```



## 6. Slow Query Alerting

---

Slow query alerts detect when query performance degrades beyond acceptable thresholds.

**Modern path (recommended for new alerting):** wire these queries into a **Davis Anomaly Detector** (configured in **Workflows**) so Davis can apply adaptive baselines, multi-dimensional analysis, and seasonal pattern detection — far more accurate than fixed thresholds for slow-query detection. See the **AIOPS** series for anomaly-detection mechanisms and the **WFLOW** series for Workflow-driven alert routing.

**Legacy path (still supported for fixed-threshold cases):** the queries below are also directly usable as **metric events** in Settings → Anomaly detection → Metric events when you want a fixed threshold instead of an adaptive baseline.

```
// Alert query: Slow query count per 5-minute window (> 500ms threshold)
fetch spans, from:-1h
| filter isNotNull(db.system)
| filter duration > 500ms
| makeTimeseries slow_count = count(), by:{db.system}, interval:5m
```

```
// Alert query: Current slow query detail – for investigation
fetch spans, from:-15m
| filter isNotNull(db.system)
| filter duration > 500ms
| fields timestamp, db.system, db.namespace, db.operation.name,
          db.query.text, server.address,
          duration_ms = duration / 1ms,
          dt.entity.service
| fieldsAdd service_name = entityName(dt.entity.service,
type:"dt.entity.service")
| sort duration_ms desc
| limit 20
```

```
// Alert query: P95 response time exceeding SLO threshold
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries p95_ms = percentile(duration / 1ms, 95),
          by:{db.system, server.address},
          interval:5m
```

## Slow Query Alert Thresholds

| Database Type                 | Warning Threshold | Critical Threshold | Window    |
|-------------------------------|-------------------|--------------------|-----------|
| SQL (PostgreSQL, MySQL, etc.) | P95 > 200ms       | P95 > 500ms        | 5 minutes |
| NoSQL (MongoDB, DynamoDB)     | P95 > 100ms       | P95 > 300ms        | 5 minutes |
| Cache (Redis, Memcached)      | P95 > 5ms         | P95 > 20ms         | 5 minutes |
| Search (Elasticsearch)        | P95 > 500ms       | P95 > 2000ms       | 5 minutes |

## 7. Database SLO Definitions

Service Level Objectives (SLOs) for databases define the expected performance contract. Use these queries to measure SLO compliance.

### Recommended Database SLOs

| SLO          | Target                              | Measurement                     |
|--------------|-------------------------------------|---------------------------------|
| Availability | 99.9% of calls succeed              | Error rate < 0.1%               |
| Latency      | 95% of calls under threshold        | P95 < vendor-specific threshold |
| Throughput   | No more than 50% drop from baseline | Queries/min vs 7-day average    |

```
// SLO measurement: Database availability – success rate over 24 hours
// span.status_code is set only on FAILED database spans (value "error"); a
// successful call
// leaves it null. `!= "error"` therefore evaluates to null, not true, and
// counts nothing –
// so successes are derived by subtraction rather than by a negative
// comparison.
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
}, by:{db.system}
| fieldsAdd successful = total - errors
| fieldsAdd availability_pct = round((toDouble(successful) / toDouble(total))
* 100, decimals:3)
| sort availability_pct asc
```

```
// SLO measurement: Latency compliance – percentage of calls under threshold
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize {
    total = count(),
    under_threshold = countIf(duration < 500ms)
}, by:{db.system}
| fieldsAdd latency_slo_pct = round((toDouble(under_threshold) /
toDouble(total)) * 100, decimals:2)
| sort latency_slo_pct asc
```

```
// SLO trend: Hourly availability over 24 hours
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries total = count(),
                  errors = countIf(span.status_code == "error", default:0),
                  by:{db.system},
                  interval:1h
```

## 8. Summary

---

In this notebook you learned:

- The key database KPIs to monitor: response time, error rate, throughput, slow queries, and connection health

- Dashboard-ready queries for a unified database health overview
- Response time monitoring with P50/P95/P99 trend analysis
- Error rate alerting queries and recommended thresholds
- Throughput monitoring including read/write ratio trends and day-over-day comparison
- Slow query alerting with vendor-specific thresholds (with Davis Anomaly Detectors as the modern path + metric events as the legacy fallback)
- Database SLO definitions and compliance measurement queries

## Series Complete

This is the final substantive notebook in the DBMON series. DBMON-99 is the consolidated best-practice summary.

For a complete database monitoring implementation:

1. **DBMON-01** — Understand fundamentals and establish baselines
2. **DBMON-02** — Deep dive into SQL database monitoring
3. **DBMON-03** — NoSQL database monitoring patterns
4. **DBMON-04** — Cache and messaging system monitoring
5. **DBMON-05** — Advanced query analysis and optimization
6. **DBMON-06** — (This notebook) Dashboards, alerting, and SLOs
7. **DBMON-99** — Best Practice Summary

## Where to Go Deeper

- **AIOPS series** (8 notebooks) — Davis AI: anomaly detection mechanisms (static / auto-adaptive / seasonal / multi-dimensional baseline / novelty/forecast), Davis problems & RCA, Davis CoPilot
- **WFLOW series** (10 notebooks) — Workflow-driven alert routing, AI tasks, MCP server integration
- **DASH series** (8 notebooks) — Dashboard strategy, executive reporting, tile-design patterns
- **FAQ-02** — Tagging strategy for ownership routing in alerts

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace*

*documentation.*